

Guida di avvio e validazione manuale

Adaptive and Policy-Aware DoS Detection and Mitigation in an SDN Network

Repository: NCIs-unina/ncis-project-work-2026-derosa-digennaro-giaquinto-gismondi

Obiettivo: questa guida accompagna un nuovo utente dall'avvio del progetto fino alla verifica pratica delle funzionalità principali: training adattivo, detection, targeted DROP, intelligent unblock, allowlist e blocklist.

Come usare questa guida

La validazione viene eseguita manualmente con tre terminali WSL separati. Ogni blocco indica chiaramente dove eseguire i comandi. Quando compare il prompt `mininet>`, eseguire soltanto comandi Mininet/host; i comandi Linux generali, i controlli sulle flow e le modifiche ai file vanno eseguiti in un normale terminale WSL.

Terminale	Contesto	Funzione
1	WSL normale	Controller Ryu
2	WSL → poi `mininet>`	Topologia e traffico degli host
3	WSL normale	Flow OpenFlow, CSV e policy JSON

Prerequisiti

- Ubuntu/WSL2 con Mininet e Open vSwitch funzionanti.
- Python 3 e `iperf3` installati.
- Ryu disponibile nel virtual environment `~/ryuenv`.
- Repository clonato localmente in `~/ncis-project`.
- Nessun'altra istanza di Mininet o Ryu attiva prima dell'avvio.

Se il repository non è ancora stato clonato:

```
cd ~
git clone https://github.com/NCIs-unina/ncis-project-work-2026-derosa-digennaro-giaquinto-gismondi.git
cd ncis-project
```

1. Verifica dell'ambiente

TERMINALE 3 — WSL normale

```
cd ~/ncis-project
git status -sb
~/ryuenv/bin/ryu-manager --version
mn --version
iperf3 --version
ovs-vsctl --version
```

Risultato atteso: il repository è accessibile e tutti i comandi richiesti rispondono senza errori.

Nota: questa sezione è solo un controllo preliminare dell'ambiente. Le spiegazioni dettagliate dei singoli comandi iniziano dalle operazioni effettive di avvio e validazione.

2. Ripristino delle policy iniziali

TERMINALE 3 — WSL normale

```
cd ~/ncis-project
python3 -c 'import json; json.dump({"blocked_ipv4":[],"allowlisted_ipv4":[]},
open("policy/blocklist.json","w"), indent=2)'
```

Risultato atteso: `policy/blocklist.json` contiene due liste vuote.

3. Cleanup iniziale di Mininet

TERMINALE 1 — WSL normale — prima di avviare Ryu

```
cd ~/ncis-project
sudo mn -c
```

Risultato atteso: Mininet rimuove eventuali residui della precedente topologia.

Importante: eseguire `sudo mn -c` prima di avviare Ryu. Non rilanciarlo mentre il controller deve restare attivo, perché il cleanup può terminare anche `ryu-manager`.

4. Avvio del controller Ryu

TERMINALE 1 — WSL normale

```
cd ~/ncis-project
source ~/ryuenv/bin/activate
RUN_ID=MANUAL_DEMO ryu-manager --ofp-tcp-listen-port 6653 controller/dos_controller.py
```

Risultato atteso: Ryu resta in esecuzione in attesa degli switch OpenFlow.

Cosa verifica: avvia il componente che esegue monitoraggio, detection, mitigazione, logging e gestione delle policy.

5. Avvio della topologia finale

TERMINALE 2 — WSL normale; poi prompt `mininet>`

```
cd ~/ncis-project
sudo python3 topology/overblocking_topo.py
```

Risultato atteso: si apre il prompt `mininet>`.

La topologia contiene cinque host e due Open vSwitch. `h1` è la sorgente usata per il traffico offensivo, `h5` è un host legittimo che condivide con `h1` l'uplink `s2 → s1`, mentre `h4` è la vittima.

6. Verifica della connettività di base

TERMINALE 2 — prompt `mininet>`

```
pingall
```

Risultato atteso: `0% dropped`.

Cosa verifica: conferma che la rete funziona correttamente prima di introdurre traffico anomalo o policy di blocco.

7. Avvio dei server iperf3

TERMINALE 2 — prompt `mininet>`

```
h4 iperf3 -s -D -p 5201
h4 iperf3 -s -D -p 5202
h4 iperf3 -s -D -p 5203
```

Risultato atteso: nessun errore; `h4` ascolta sulle tre porte utilizzate dai flussi di prova.

8. Training benigno e threshold adattiva

TERMINALE 2 — prompt `mininet>`

```
h5 iperf3 -c 10.0.0.4 -u -b 2M -t 15 -p 5202
```

Attendere il completamento del comando. Il controller usa questi campioni per costruire la baseline iniziale.

TERMINALE 3 — WSL normale

```
cd ~/ncis-project
tail -n 10 results/raw/MANUAL_DEMO_controller.csv
```

Risultato atteso: nelle righe più recenti compaiono `baseline\_mbps`, `threshold\_mbps`, `training\_count` e `trained`. Con questo traffico la baseline dovrebbe essere circa 2 Mbit/s e la threshold circa 3 Mbit/s.

Cosa verifica: verifica che il detector apprenda il traffico benigno e ricavi una soglia adattiva.

9. Generazione dell'attacco e targeted DROP

Si mantengono contemporaneamente un flusso benigno da `h5` e un flusso ad alto rate da `h1`. Entrambi attraversano l'uplink condiviso `s2 → s1`.

TERMINALE 2 — prompt `mininet>`

```
h5 sh -c 'iperf3 -c 10.0.0.4 -u -b 2M -t 70 -p 5202 > /tmp/manual_h5_legit.log 2>&1 & echo $! > /tmp/manual_h5_legit.pid'
h1 sh -c 'iperf3 -c 10.0.0.4 -u -b 20M -t 70 -p 5201 > /tmp/manual_h1_attack.log 2>&1 & echo $! > /tmp/manual_h1_attack.pid'
```

Attendere circa 8–10 secondi per consentire al detector di osservare più campioni sopra soglia.

TERMINALE 3 — WSL normale

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1 | grep 'priority=100' | grep 'nw_src=10.0.0.1'
grep 'DROP_INSTALLED' results/raw/MANUAL_DEMO_controller.csv | tail -n 1
```

Risultato atteso: compare una flow `priority=100` con `nw\_src=10.0.0.1`; nel CSV compare `DROP\_INSTALLED`.

Cosa verifica: verifica che il controller installi un DROP mirato all'IPv4 sorgente dell'attaccante.

10. Verifica dell'assenza di over-blocking

TERMINALE 2 — prompt `mininet>`

```
h1 ping -c 4 -W 1 10.0.0.4
h5 ping -c 4 -W 1 10.0.0.4
h2 ping -c 4 -W 1 10.0.0.4
```

Risultato atteso: `h1` circa 100% packet loss; `h5` e `h2` 0% packet loss.

Cosa verifica: dimostra che il blocco colpisce solo `h1`, mentre `h5`, pur condividendo lo stesso uplink, resta raggiungibile.

11. Persistenza oltre i vecchi 20 secondi

Lasciare l'attacco di `h1` attivo e attendere almeno 22 secondi dopo l'installazione del DROP.

TERMINALE 3 — WSL normale

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1 | grep 'priority=100' | grep 'nw_src=10.0.0.1'
```

Risultato atteso: la flow per `10.0.0.1` è ancora presente.

Cosa verifica: dimostra che il blocco resta attivo finché il traffico offensivo continua.

12. Intelligent Unblock

Fermare soltanto il traffico offensivo di `h1`. Non cancellare manualmente la flow OpenFlow.

TERMINALE 2 — prompt `mininet>`

```
h1 sh -c 'kill $(cat /tmp/manual_h1_attack.pid) 2>/dev/null || true'
```

Attendere circa 6–8 secondi.

TERMINALE 3 — WSL normale

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1 | grep 'priority=100' | grep 'nw_src=10.0.0.1'
grep 'DROP_REMOVED' results/raw/MANUAL_DEMO_controller.csv | tail -n 1
```

Risultato atteso: il primo comando non produce più output e nel CSV compare `DROP\_REMOVED`.

Cosa verifica: verifica che sia il controller, osservando la sorgente quieta, a rimuovere il DROP.

TERMINALE 2 — prompt `mininet>`

```
h1 ping -c 4 -W 1 10.0.0.4
h5 ping -c 4 -W 1 10.0.0.4
h2 ping -c 4 -W 1 10.0.0.4
h5 sh -c 'kill $(cat /tmp/manual_h5_legit.pid) 2>/dev/null || true'
```

Risultato atteso: `h1`, `h5` e `h2` sono nuovamente raggiungibili.

13. Allowlist runtime: host trusted

TERMINALE 3 — WSL normale

```
cd ~/ncis-project
python3 -c 'import json; json.dump({"blocked_ipv4":[],"allowlisted_ipv4":["10.0.0.5"]},
open("policy/blocklist.json","w"), indent=2)'
```

Attendere circa 2 secondi affinché il controller rileggi il file di policy.

TERMINALE 2 — prompt `mininet>`

```
h5 sh -c 'iperf3 -c 10.0.0.4 -u -b 20M -t 40 -p 5203 > /tmp/manual_h5_high.log 2>&1 & echo $! >
/tmp/manual_h5_high.pid'
```

Attendere circa 8–10 secondi.

TERMINALE 3 — WSL normale

```
grep 'ALLOWLIST_SUPPRESSED' results/raw/MANUAL_DEMO_controller.csv | tail -n 1
sudo ovs-ofctl -O OpenFlow13 dump-flows s1 | grep 'priority=100' | grep 'nw_src=10.0.0.5'
```

Risultato atteso: il CSV mostra `ALLOWLIST\_SUPPRESSED`; non compare alcuna flow automatica per `h5`.

Cosa verifica: verifica che un host trusted continui a essere monitorato ma non venga mitigato automaticamente.

14. Allowlist OFF con lo stesso traffico

Lasciare il flusso alto di `h5` attivo e rimuovere `h5` dall'allowlist.

TERMINALE 3 — WSL normale

```
python3 -c 'import json; json.dump({"blocked_ipv4":[],"allowlisted_ipv4":[]},
open("policy/blocklist.json","w"), indent=2)'
```

Risultato atteso: compare il targeted DROP automatico per `10.0.0.5`.

Cosa verifica: mostra che, a parità di host e traffico, la policy allowlist modifica l'esito della mitigazione.

TERMINALE 2 — prompt `mininet>`

```
h5 ping -c 4 -W 1 10.0.0.4
h5 sh -c 'kill $(cat /tmp/manual_h5_high.pid) 2>/dev/null || true'
```

Risultato atteso: durante il DROP `h5` mostra circa 100% packet loss.

Dopo aver fermato il flusso alto, attendere 6–8 secondi per consentire l'unblock automatico.

15. Blocklist amministrativa ON

TERMINALE 3 — WSL normale

```
python3 -c 'import json; json.dump({"blocked_ipv4":["10.0.0.5"],"allowlisted_ipv4":[]},
open("policy/blocklist.json","w"), indent=2)'
```

Risultato atteso: compare una flow `priority=110`, `in_port=3`, `nw_src=10.0.0.5`, `actions=drop` su `s2`.

Cosa verifica: verifica che una decisione esterna venga trasformata a runtime in una regola OpenFlow persistente.

TERMINALE 2 — prompt `mininet>`

```
h5 ping -c 4 -W 1 10.0.0.4
h1 ping -c 4 -W 1 10.0.0.4
h2 ping -c 4 -W 1 10.0.0.4
```

Risultato atteso: `h5` circa 100% packet loss; `h1` e `h2` 0%.

16. Blocklist amministrativa OFF

TERMINALE 3 — WSL normale

```
python3 -c 'import json; json.dump({"blocked_ipv4":[],"allowlisted_ipv4":[]},
open("policy/blocklist.json","w"), indent=2)'
```

Risultato atteso: nessun output: la flow amministrativa è stata rimossa.

TERMINALE 2 — prompt `mininet>`

```
h5 ping -c 4 -W 1 10.0.0.4
```

Risultato atteso: `h5` torna raggiungibile con 0% packet loss.

17. Stato finale e file prodotti

TERMINALE 3 — WSL normale

```
cd ~/ncis-project
tail -n 20 results/raw/MANUAL_DEMO_controller.csv
cat policy/blocklist.json
```

Cosa fanno i comandi:

- `tail -n 20 ...` — mostra gli ultimi eventi della sessione appena eseguita, utili per controllare `DROP_INSTALLED`, `DROP_REMOVED`, `ALLOWLIST_SUPPRESSED` e i valori del detector.

Risultato atteso: il CSV contiene gli eventi del test e il file JSON mostra entrambe le liste vuote.

18. Cleanup

TERMINALE 2 — prompt `mininet>`

```
exit
```

TERMINALE 1 — Ryu

Interrompere il controller con:

```
Ctrl+C
```

Risultato atteso: nessun processo Mininet/OVS residuo.

19. Risoluzione dei problemi più comuni

Problema	Controllo / soluzione
<code>ryu-manager</code> non parte	Verificare <code>~/ryuenv</code> , chiudere eventuali vecchie istanze Ryu e assicurarsi che la porta 6653 non sia già occupata.
La topologia parte ma gli switch non comunicano	Controllare che Ryu sia stato avviato prima di

	`overblocking_topo.py` e che ascolti sulla porta TCP 6653.
Una flow non compare subito	Attendere 8–10 secondi: il controller campiona periodicamente e richiede più campioni consecutivi.
Il grep di una flow non stampa nulla durante Allowlist ON	È il comportamento corretto: la mitigazione automatica di `h5` deve essere soppressa.
`MANUAL_DEMO_controller.csv` non esiste	Verificare che Ryu sia stato avviato con `RUN_ID=MANUAL_DEMO` e che il controller abbia ricevuto statistiche dagli switch.
I valori live non coincidono esattamente con quelli della relazione	È normale: timing, rate istantanei e polling dipendono dall'allineamento temporale e dal carico della macchina.